

RNN Study Guide

Recurrent Neural Networks

Applied Artificial Intelligence · Hof University · Prof. Dr. Lars Ackermann-Igl

This guide covers every concept from the RNN lecture slides. Each question has three levels: (1) a plain-English explanation so you truly understand it, (2) worked examples with real numbers and equations, and (3) a highlighted **Exam-Ready Answer** box showing exactly what to write under time pressure. Mock exam tasks have been analysed — all likely question types are covered.

Part	Topic
Part 1	Why RNNs? MLPs, CNNs & Sequential Data
Part 2	RNN Basics — Hidden State & Recurrence Formula
Part 3	Elman Networks — Forward Pass & Inference
Part 4	Counting Trainable Parameters (Formula + Examples)
Part 5	Backpropagation Through Time (BPTT)
Part 6	Vanishing Gradient Problem — Causes & Solution
Part 7	Long Short-Term Memory (LSTM)
Part 8	Implementing RNNs in Keras
Part 9	Mock Exam Worked Questions (Task 5 Complete)
Part 10	Final Cheat Sheet

PART 1 — Why RNNs? The Problem with Sequential Data

Q1. What is the essential difference between MLPs/CNNs and RNNs?

MLPs and CNNs are both **feed-forward neural networks**. This means information flows in one direction only — from input layer through hidden layers to output layer. Connections only exist between layer l and layer $l+1$. There are no loops, no self-connections, no connections going backwards.

This design has one critical limitation: **every input is processed completely independently**. The network has no memory. When it processes input at time step t , it has no knowledge of what inputs it saw at steps $t-1$, $t-2$, etc. This is fine for images (a single photo processed at once), but breaks down completely for sequential data.

Examples of sequential data:

- Text / language — the meaning of 'bank' depends on surrounding words ('river bank' vs 'bank account')
- Time series — stock prices, sensor readings, weather data where past values predict the future
- Speech — audio where what came before determines what you hear now
- Video — frames that only make sense in temporal context

■ EXAM-READY ANSWER

MLPs and CNNs are feed-forward networks: connections only exist between layer l and $l+1$. They process each input independently with no memory of previous inputs.

RNNs differ by introducing **recurrent connections** (loops / self-references) within the hidden layer. This allows the network to maintain an internal **hidden state** that is updated at each time step, carrying information from previous steps into the current calculation. This enables processing of **variable-length sequential inputs**.

Q2. What types of input/output configurations do RNNs support?

Because RNNs process sequences step-by-step, they are flexible in how many inputs they consume and how many outputs they produce:

Configuration	Inputs	Outputs	Example
One-to-One	Single	Single	Standard MLP (not really RNN use-case)
One-to-Many	Single	Sequence	Image captioning — 1 image → many words
Many-to-One	Sequence	Single	Sentiment analysis — many words → 1 label
Many-to-Many (same)	Sequence	Sequence (same length)	Video frame labelling
Many-to-Many (diff)	Sequence	Sequence (different length)	Machine translation

PART 2 — RNN Basics: Hidden State & Recurrence Formula

Q3. What is the core idea behind an RNN and what is the recurrence formula?

The key innovation of an RNN is an **internal hidden state** — a vector of numbers that acts as the network's memory. It is updated at every time step to incorporate both the current input and the memory from all previous time steps.

This is expressed as a **recurrence relation** (recursive function):

$$h_t = f_W(h_{t-1}, x_t)$$

Where:

h_t = new hidden state (updated memory) at time step t

h_{t-1} = old hidden state from the previous time step

x_t = current input vector at time step t

f_W = some function parameterised by weight matrix W

Key point: The *same* function f_W with the *same* weights W is applied at *every* time step. Only the inputs (h_{t-1} and x_t) change. This is called **weight sharing across time** and is what makes RNNs efficient.

■ EXAM-READY ANSWER — Recursive Function

The processing principle of an RNN is expressed as the recurrence relation:

$$h_t = f_W(h_{t-1}, x_t)$$

where h_t is the new hidden state, h_{t-1} is the previous hidden state, x_t is the current input, and f_W is a function with trainable parameters W . The same W is used at every time step. The initial state h_0 is typically set to the zero vector.

Q4. How does an RNN differ from an MLP in terms of the gradient descent method?

In an MLP, activation functions like ReLU or sigmoid are used, which are differentiable everywhere (or almost everywhere). This allows gradient descent via standard backpropagation.

In RNNs, the architecture creates **cycles / loops** in the computation graph. Standard backpropagation through a static network graph is not sufficient — we must use **Backpropagation Through Time (BPTT)**, which unrolls the RNN across all time steps and then applies backpropagation through this unrolled graph. This is more complex and introduces the vanishing/exploding gradient problem (see Part 6).

PART 3 — Elman Networks: Forward Pass & Inference

Q5. What is an Elman RNN and what are its exact equations?

The **Elman network** (also called Simple RNN or Vanilla RNN) is the most basic form of RNN. It uses a linear combination of the hidden state and current input, passed through a **tanh activation function**.

Hidden state update:

$$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t + b_h)$$

Output calculation:

$$y_t = W_{hy} \cdot h_t + b_y$$

Where:

W_{hh} = hidden-to-hidden weight matrix ($n \times n$)

W_{xh} = input-to-hidden weight matrix ($n \times m$)

W_{hy} = hidden-to-output weight matrix ($o \times n$)

b_h = hidden bias vector ($n \times 1$)

b_y = output bias vector ($o \times 1$)

n = number of hidden neurons

m = number of input features

o = number of output neurons

$\tanh$ = hyperbolic tangent $\rightarrow \tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$

■ Why tanh?

$\tanh$ maps any value to the range $(-1, +1)$. This prevents the hidden state values from growing without bound as the sequence is processed. The $\tanh$ derivative is: $\tanh'(x) = 1 - \tanh^2(x)$, which is always between 0 and 1 — this is important for the vanishing gradient discussion later.

Q6. Worked Example — Forward Pass Step by Step

This is the kind of detailed calculation that can appear in an exam. Follow every step carefully.

GIVEN:

Inputs: $x_1 = 4$, $x_2 = -1$

Weights: $w_{xh} = 1$, $w_{hh} = 2$, $w_{hy} = 3$

Biases: $b_h = -1$, $b_y = 1$

Initial state: $h_0 = 0$

FORWARD PASS EQUATIONS:

$h_t = \tanh(w_{hh} \cdot h_{t-1} + w_{xh} \cdot x_t + b_h)$

$y_t = w_{hy} \cdot h_t + b_y$

$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$

TIME STEP 1 ($x_1 = 4$):

$h_1 = \tanh(w_{hh} \cdot h_0 + w_{xh} \cdot x_1 + b_h)$

$= \tanh(2 \cdot 0 + 1 \cdot 4 + (-1))$

$= \tanh(0 + 4 - 1)$

$= \tanh(3)$

≈ 0.995

TIME STEP 2 ($x_2 = -1$):

$h_2 = \tanh(w_{hh} \cdot h_1 + w_{xh} \cdot x_2 + b_h)$

$= \tanh(2 \cdot 0.995 + 1 \cdot (-1) + (-1))$

$= \tanh(1.99 - 1 - 1)$

$= \tanh(-0.01)$

≈ -0.01

OUTPUT at $t=2$:

$y_2 = w_{hy} \cdot h_2 + b_y$

$= 3 \cdot (-0.01) + 1$

$= -0.03 + 1$

$= 0.97$

■ EXAM-READY ANSWER — Forward Pass

Step 1: Compute $h_1 = \tanh(w_{hh} \cdot h_0 + w_{xh} \cdot x_1 + b_h)$ — always start from $h_0 = 0$.

Step 2: Plug h_1 into the equation for h_2 — note that the PREVIOUS hidden state flows in.

Step 3: Compute the output: $y_t = w_{hy} \cdot h_t + b_y$.

Key insight: h_t is a summarisation of ALL inputs seen up to time step t .

PART 4 — Counting Trainable Parameters

Q7. How do you calculate the total number of trainable parameters in an Elman RNN?

This is one of the highest-likelihood exam calculation questions. The formula comes directly from the three weight matrices:

PARAMETER COUNT FORMULA (without bias):

$$N_{\text{params}} = n^2 + n \cdot m + o \cdot n$$

Where:

n = number of hidden neurons

m = number of input features (input size)

o = number of output neurons (output size)

Breaking it down:

$n^2 = W_{hh}$ (hidden-to-hidden): n rows $\times$ n columns

$n \cdot m = W_{xh}$ (input-to-hidden): n rows $\times$ m columns

$o \cdot n = W_{hy}$ (hidden-to-output): o rows $\times$ n columns

WITH BIAS NEURONS ADD:

+ n (bias for hidden layer: b_h)

+ o (bias for output layer: b_y)

Total WITH bias = $n^2 + n \cdot m + o \cdot n + n + o$

Why n^2 for W_{hh} ?

The hidden-to-hidden matrix W_{hh} connects every hidden neuron at time step $t-1$ to every hidden neuron at time step t . With n hidden neurons, this is an $n \times n$ matrix $\rightarrow n^2$ weights. This is the key architectural insight that makes RNNs different from feed-forward networks: the hidden layer connects to itself.

Worked Example (from Mock Exam Task 5c):

Elman RNN: input size = 10, one hidden layer of size 20, output size = 5. No bias neurons.

$n = 20$ (hidden neurons)

$m = 10$ (input size)

$o = 5$ (output size)

$W_{hh} = n^2 = 20^2 = 400$

$W_{xh} = n \cdot m = 20 \cdot 10 = 200$

$W_{hy} = o \cdot n = 5 \cdot 20 = 100$

TOTAL = $400 + 200 + 100 = 700$ parameters

Additional Example (from lecture slides, without bias):

$n = 4, m = 3, o = 2$

$N_{\text{params}} = 4^2 + 4 \cdot 3 + 2 \cdot 4 = 16 + 12 + 8 = 36$

■ EXAM-READY ANSWER — Parameter Count

Formula (no bias): $N_{\text{params}} = n^2 + n \cdot m + o \cdot n$

Formula (with bias): $N_{\text{params}} = n^2 + n \cdot m + o \cdot n + n + o$

For Mock Exam Task 5c (input=10, hidden=20, output=5, no bias):

$N_{\text{params}} = 20^2 + 20 \cdot 10 + 5 \cdot 20 = 400 + 200 + 100 = 700$

Always state which weight matrix each term comes from (W_{hh} , W_{xh} , W_{hy}) for full marks.

PART 5 — Backpropagation Through Time (BPTT)

Q8. What is BPTT and why is it needed for RNNs?

Standard backpropagation works on a static, directed acyclic graph. An RNN has **recurrent connections** (cycles), which break this requirement. To apply backpropagation, we **unroll** the RNN through time — we create a copy of the network for each time step, connecting them in sequence. This turns the cyclic graph into an acyclic one that backpropagation can handle.

The BPTT procedure:

1. **Forward Pass:** Process the ENTIRE input sequence, computing h_t and y_t at each step. Store ALL intermediate values (needed for gradients).
2. **Compute Loss:** Calculate the loss after the full sequence (or after each time step).
3. **Backward Pass:** Backpropagate errors through the ENTIRE unrolled sequence — both through layers (like standard backprop) AND back through time steps.
4. **Update weights:** Accumulate gradients across all time steps, then update W_{hh} , W_{xh} , W_{hy} .

■ Truncated BPTT

For very long sequences, full BPTT is computationally expensive and memory-intensive. Truncated BPTT splits the sequence into smaller chunks and runs BPTT on each chunk separately. This is commonly used in practice. It is also justified because standard (Elman) RNNs struggle with very long-range dependencies anyway (see Part 6).

PART 6 — Vanishing Gradient Problem

Q9. What is the vanishing gradient problem and what causes it in Elman RNNs?

During BPTT, gradients must be propagated backwards through many time steps. At each step, the gradient is multiplied by the partial derivative of h_t with respect to h_{t-1} :

$$\partial h_t / \partial h_{t-1} = \tanh'(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t) \cdot W_{hh}$$

where $\tanh'(x) = 1 - \tanh^2(x) \rightarrow$ always in the range $(0, 1]$

For the full gradient from time T back to time 1:

$$\partial L_T / \partial W = (\partial L_T / \partial h_T) \cdot \text{PRODUCT}_{t=2}^T (\partial h_t / \partial h_{t-1}) \cdot (\partial h_1 / \partial W)$$

The problem: The product term multiplies $T-1$ factors together, each of which is between 0 and 1 (because $\tanh'$ is always ≤ 1). Multiplying many small numbers together causes the product to shrink *exponentially* toward zero:

$$0.9^{10} \approx 0.35 \text{ (after 10 steps – still some signal)}$$

$$0.9^{50} \approx 0.005 \text{ (after 50 steps – nearly gone)}$$

$$0.5^{20} \approx 0.000001 \text{ (with smaller factors – vanishes very fast)}$$

Consequence: Gradients reaching early time steps are essentially zero. The weights at early time steps get almost no update signal — the network *cannot learn dependencies that span many time steps*. This is why Elman RNNs cannot model long-term dependencies well.

General causes of the vanishing gradient problem:

1. **tanh activation in Elman RNNs** — derivative always ≤ 1 , leading to exponential shrinkage when multiplied across many steps.
2. **Deep networks (many layers)** — the same multiplication-of-small-values effect occurs in very deep feed-forward networks with sigmoid/tanh activation.
3. **Large number of time steps** — even if individual gradients are only slightly less than 1, multiplying across hundreds of steps drives the product to zero.

■ EXAM-READY ANSWER — Vanishing Gradient

General meaning: during backpropagation, gradients are multiplied by partial derivatives at each layer/time step. If these derivatives are consistently < 1 , the gradient shrinks exponentially as it propagates backwards, eventually becoming effectively zero.

Specific cause in Elman RNNs: the tanh activation function has derivative $\tanh'(x) = 1 - \tanh^2(x) \in (0, 1]$. During BPTT, the gradient is multiplied by this value at every time step. Multiplying many values < 1 together causes exponential decay $\rightarrow$ weights at early time steps receive no meaningful gradient $\rightarrow$ the network cannot learn long-range dependencies.

Another general cause: sigmoid activation in deep feed-forward networks causes the same problem across layers.

PART 7 — Long Short-Term Memory (LSTM)

Q10. Why was LSTM invented and what is its key idea?

Because the vanishing gradient problem is fundamental to the tanh-based Elman architecture, there is essentially no way to fix it within the Elman framework. Hochreiter & Schmidhuber introduced the **Long Short-Term Memory (LSTM)** architecture in 1997 to address this.

Key idea: Add a second memory pathway called the **cell state** (c_t), separate from the hidden state (h_t). The cell state can carry information across many time steps with minimal modification — providing an **uninterrupted gradient flow** that allows gradients to propagate back without exponential decay.

The three gates that control the cell state:

Gate	Symbol	Function	What it controls
Forget Gate	f	$\sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$	How much of c_{t-1} to FORGET (keep)
Input Gate	i	$\sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$	How much new info to ADD to cell state
Output Gate	o	$\sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$	How much of cell state to OUTPUT as h_t
Candidate	g	$\tanh(W_g \cdot [h_{t-1}, x_t] + b_g)$	New candidate values to add to cell state

LSTM Update Equations:

$(i, f, o, g) = (\sigma, \sigma, \sigma, \tanh) \cdot W \cdot [h_{t-1}, x_t]$ (one joint matrix multiply)

$c_t = f \blacksquare c_{t-1} + i \blacksquare g$ (update cell state)

$h_t = o \blacksquare \tanh(c_t)$ (update hidden state)

$\blacksquare$ = element-wise (Hadamard) product

σ = sigmoid function $\rightarrow$ output in $(0, 1)$

■ Why does LSTM help with vanishing gradients?

The cell state c_t is updated via ADDITION ($c_t = f \blacksquare c_{t-1} + i \blacksquare g$) rather than multiplication through tanh. Addition creates an uninterrupted gradient highway — gradients can flow backwards through the cell state path with much less decay. Note: LSTMs reduce but do not completely eliminate the vanishing gradient problem. They are, however, much more capable of learning long-range dependencies than Elman RNNs.

PART 8 — Implementing RNNs in Keras

Q11. How do you implement an Elman RNN in Keras?

```

model = keras.Sequential([
    layers.SimpleRNN(
        units=8, # number of hidden neurons (n)
        input_shape=(3, 1), # (time_steps, features_per_step)
        return_sequences=False # False = many-to-one, True = many-to-many
    ),
    layers.Dense(1) # output layer: maps hidden state to output
])

```

Key parameters explained:

Parameter	Meaning	Typical Values
units	Number of hidden neurons (n)	8, 16, 32, 64, 128, ...
input_shape	(time_steps, features_per_step)	e.g. (3,1) = 3 steps, 1 feature each
return_sequences=False	Only return h _T (last hidden state)	Use for many-to-one (classification)
return_sequences=True	Return h _t for every time step	Use for many-to-many (e.g. translation)
SimpleRNN	Implements Elman RNN (tanh default)	Also: LSTM, GRU layers available
Dense(1)	Maps final hidden state to output	Change units for multi-output tasks

PART 9 — Mock Exam Worked Questions (Task 5 Complete)

Task 5a — What difference allows RNNs to process sequential data? Express as recursive function. (3P)

■ COMPLETE ANSWER (3 points)

Key difference: Unlike MLPs and CNNs (which are feed-forward and have no memory), RNNs introduce recurrent connections in the hidden layer. This gives RNNs an internal hidden state that is updated at each time step, allowing information from previous time steps to influence current processing.

Recursive function:

$$h_t = f_W(h_{t-1}, x_t)$$

where h_t is the new hidden state at time t , h_{t-1} is the hidden state from the previous time step (the 'memory'), x_t is the current input, and f_W is a function with trainable parameters W applied identically at every time step.

For an Elman RNN specifically:

$$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t + b_h)$$

Task 5b — Vanishing Gradient Problem: general meaning, Elman cause, one other cause. (4P)

■ COMPLETE ANSWER (4 points)

General meaning of vanishing gradient:

During backpropagation, the gradient of the loss with respect to early weights is computed by multiplying partial derivatives through each layer/time step. When these derivatives are consistently less than 1, the gradient shrinks exponentially, approaching zero for early layers/time steps. As a result, early weights receive almost no update signal and the network fails to learn long-range dependencies.

Specific property of Elman RNNs that causes it:

Elman RNNs use the tanh activation function, whose derivative is $\tanh'(x) = 1 - \tanh^2(x)$, always in the range (0, 1]. During BPTT, the gradient at time T must be multiplied by this factor at every preceding time step. Multiplying many values ≤ 1 together causes exponential decay: $(0.9)^{100} \approx 0.00003$. This means gradients vanish rapidly for long sequences.

One other general cause:

The vanishing gradient problem also occurs in deep feed-forward networks that use sigmoid or tanh activations, since these functions also have derivatives ≤ 1 (sigmoid max derivative = 0.25).

Task 5c — Parameter count: input=10, hidden=20, output=5, no bias. (2P)

■ COMPLETE ANSWER (2 points)

Using the formula $N_{\text{params}} = n^2 + n \cdot m + o \cdot n$ (without bias):

$n = 20$ (hidden neurons)

$m = 10$ (input size)

$o = 5$ (output size)

$W_{hh} = n^2 = 20 \times 20 = 400$

$W_{xh} = n \cdot m = 20 \times 10 = 200$

$W_{hy} = o \cdot n = 5 \times 20 = 100$

TOTAL = $400 + 200 + 100 = 700$ trainable parameters

Additional Likely Exam Question: Describe the forward pass of an Elman RNN at multiple time steps with given values.

■ Common Exam Traps

1. h_0 is ALWAYS zero initially (unless told otherwise) — don't invent a non-zero h_0 .
2. The SAME W_{hh} , W_{xh} , W_{hy} are used at EVERY time step — they do not change per step.
3. tanh is applied to the ENTIRE expression inside — not just part of it.
4. The output y_t uses the CURRENT h_t , not the previous h_{t-1} .
5. For parameter counting: n^2 comes from W_{hh} ($n \times n$ matrix) — don't forget to square it.
6. Bias neurons: the exam question will say 'do not include bias' or 'include bias' — read carefully!

Additional Likely: Compare Elman RNN to MLP and CNN.

Criterion	MLP	CNN	Elman RNN
Architecture	Feed-forward, FC layers	Feed-forward, conv/pool layers	Recurrent (has loops)
Input type	Fixed-size 1D vector	3D volume (image)	Variable-length sequence
Memory	None	None	Hidden state h_t
Feature extraction	Manual / hand-crafted	Automatic (convolutions)	Sequential patterns via h_t
Suitable for	Tabular data	Images	Sequential/time-series data
Long-term deps	No	No	Limited (vanishing grad)
Parameter sharing	No	Yes (spatial)	Yes (across time steps)

PART 10 — Final Cheat Sheet (Everything on One Page)

Concept	Key Fact / Formula
Feed-forward vs RNN	FF: connections only $l \rightarrow l+1$. RNN: recurrent connections within hidden layer
RNN use case	Sequential data: text, time series, audio, video — variable-length inputs
Recurrence formula	$h_t = f_W(h_{t-1}, x_t)$ — same W used at every time step
Elman hidden state	$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t + b_h)$
Elman output	$y_t = W_{hy} \cdot h_t + b_y$
W_{hh} size	$n \times n$ (hidden-to-hidden, where n = hidden neurons)
W_{xh} size	$n \times m$ (input-to-hidden, where m = input size)
W_{hy} size	$o \times n$ (hidden-to-output, where o = output size)
Params (no bias)	$N = n^2 + n \cdot m + o \cdot n$
Params (with bias)	$N = n^2 + n \cdot m + o \cdot n + n + o$
Example: in=10,h=20,out=5	$N = 400 + 200 + 100 = 700$ (no bias)
tanh derivative	$\tanh'(x) = 1 - \tanh^2(x) \rightarrow$ always in $(0, 1]$
tanh formula	$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$
BPTT	Unroll RNN across time steps, apply backprop through entire unrolled graph
Truncated BPTT	Split long sequences into chunks; practical solution for long sequences
Vanishing gradient (general)	Gradient = product of many derivatives $< 1 \rightarrow$ exponential decay $\rightarrow 0$
Vanishing gradient (Elman)	$\tanh' \leq 1$ multiplied T times across BPTT $\rightarrow$ gradients vanish for early steps
Cannot learn	Long-range dependencies — Elman RNN has poor gradient flow for long sequences
LSTM cell state	$c_t = f \odot c_{t-1} + i \odot g$ — addition creates uninterrupted gradient highway
LSTM hidden state	$h_t = o \odot \tanh(c_t)$
LSTM gates	Forget(f), Input(i), Output(o) — all use sigmoid $\sigma \rightarrow$ output $(0,1)$
LSTM vs Elman	LSTM: much better at long-range deps; no guarantee VGP absent but far less likely
Keras SimpleRNN	<code>layers.SimpleRNN(units=n, input_shape=(timesteps, features))</code>
<code>return_sequences=False</code>	Output only last h_T — use for many-to-one (classification, regression)
<code>return_sequences=True</code>	Output h_t at every step — use for many-to-many (translation, labelling)
h_0 initialisation	Typically zero vector — $h_0 = 0$
Weight sharing (time)	W_{hh}, W_{xh}, W_{hy} are IDENTICAL at every time step — greatly reduces params

Concept	Key Fact / Formula
Alternative: GRU	Gated Recurrent Unit — similar to LSTM but fewer parameters
Alternative: Transformer	Attention mechanism; processes sequences in parallel; state of the art (GPT, BERT)

Good luck in the exam! Remember: always show your working clearly, write the formula before substituting numbers, and label which weight matrix each parameter count term comes from. The professor awards marks for correct reasoning, not just the final number.